RAMBØLL

Structural AI Toolchain

1-page starter — everything runs locally from `structural-ai-toolchain/`. AI optional: set `ANTHROPIC_API_KEY`.

install once:
`pip install -r requirements.txt`

1 Put a sheet IN → get a tool OUT

```
python main.py ingest sheet.mcdx
python main.py gui
python main.py grind sheet.xlsx
```

ingest is the thorough way: it breaks the file into pieces, looks up the code clauses, **asks you** what to keep, which case governs and what the final UR is — then rebuilds and loops until it converges. .mcdx .xlsx .xlsm .csv .ipynb .py.

2 Already exists? → the knowledge base asks

```
revise <tool> · new tool · abort
```

Before anything is written the central library is checked for duplicates and for formulas it already holds (those keep their clause and are reused, not re-typed).

3 Double-check → the master reviewer

```
python main.py review <tool>
python main.py review <tool> --fix
```

Units, dimensions, power-of-ten slips, values vs the original (`--original print.pdf`). It **fixes** what it can prove; corrected steps get a yellow note.

4 If it asks questions → answer

```
repository/<tool>/CLARIFICATIONS.md
```

e.g. “Cell H1 = 1.0 has no label — what is it?”. It never silently guesses; open points also sit in `JOURNAL.md`.

5 See the result → the report

```
python main.py render <tool> --pdf
python main.py render <tool> --pdf --mode compact
```

A4, logo, title block, every formula written out with numbers and units, clause in the margin, OK / NOT OK.

6 Print / share → normal PDF

```
reconstruct_from_pdf("file.pdf")
```

Ctrl+P as usual. The whole calculation also hides *inside* the page, so the PDF alone can rebuild it later.

7 Change inputs → edit 1 file

```
repository/<tool>/<tool>.py
```

Edit the numbers in `build_sheet()`, render again. Every value recomputes — never hand-typed.

8 Build a NEW tool from scratch

```
python main.py new "Punching Shear (EC2 6.4)"
python main.py register <slug>
```

Starter file → edit → register. Reuse library formulas with `sheet.apply()`.

9 Find a formula to reuse

```
python main.py atoms --search "crack width"
python main.py atoms --tree
```

Drill down country → code → chapter → clause, or call it: `custom.E_cm(f_cm=38)`.

10 See EVERYTHING (non-coders)

```
python main.py dashboard
python main.py doctor
```

What's hosted, reused, reviewed and who edited it. **doctor** shows whether the AI and the PDF printer are set up.

Where things live: `repository/` = your tools + reports (the source of truth) · `src/library/` = shared formulas · `docs/` = deeper guides & PITFALLS. **Full run once:** `python main.py demo`. **Does it exist already?** `python main.py ask "what you want"`.